CORRIGÉ · EXERCICE · INFORMATIQUE MPSI

Recherche dichotomique

Énoncé

Écrire une fonction `appartient(L, x)` qui teste si `x` est dans la liste triée `L` par dichotomie. Prouver sa terminaison et donner sa complexité.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Une liste `L` triée dans l'ordre croissant, de longueur n.

Ce qu'on te demande. Un code correct, une preuve de terminaison, une complexité.

Corrigé

1. Le code

```
def appartient(L, x):
    g, d = 0, len(L) - 1
    while g <= d:
        m = (g + d) // 2
        if L[m] == x:
            return True
        elif L[m] < x:
            g = m + 1
        else:
            d = m - 1
    return False
```

2. Invariant

Si `x` est dans `L`, alors il est dans `L[g..d]`. Chaque mise à jour n'écarte qu'une zone où `x` ne peut pas être, car `L` est triée.

3. Terminaison

$d - g + 1$ est un entier positif tant que la boucle tourne, et il décroît strictement à chaque tour.

4. Complexité

La zone de recherche est au moins divisée par 2 à chaque tour : au plus $\lfloor \log_2 n \rfloor + 1$ tours, soit $O(\log n)$.

Le piège

Écrire `g = m` au lieu de `g = m + 1` : la boucle peut ne jamais terminer.

La méthode

Pour toute boucle `while`, donne un variant pour la terminaison et un invariant pour la correction.

Rappels de cours utilisés

- **Invariant de boucle.** Une propriété vraie avant la boucle et conservée par chaque tour. À la sortie, elle prouve la correction de l'algorithme.
- **Variant de boucle.** Un entier positif qui décroît strictement à chaque tour. Son existence prouve que la boucle termine.
- **Complexité logarithmique.** Si la taille du problème est au moins divisée par 2 à chaque étape, le nombre d'étapes est au plus $\lfloor \log_2 n \rfloor + 1$, soit $O(\log n)$.